

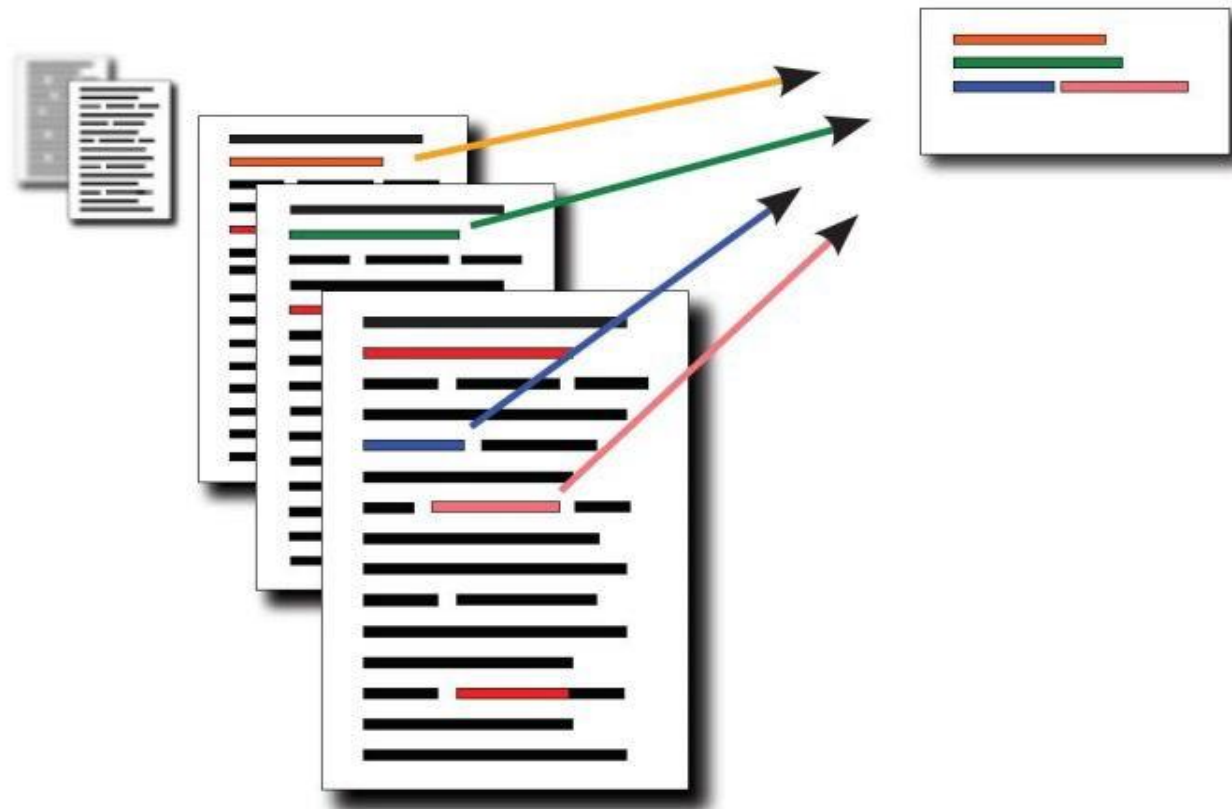
Text Summarization

Lê Thanh Hương

School of Information and Communication Technology

What is text summarization?

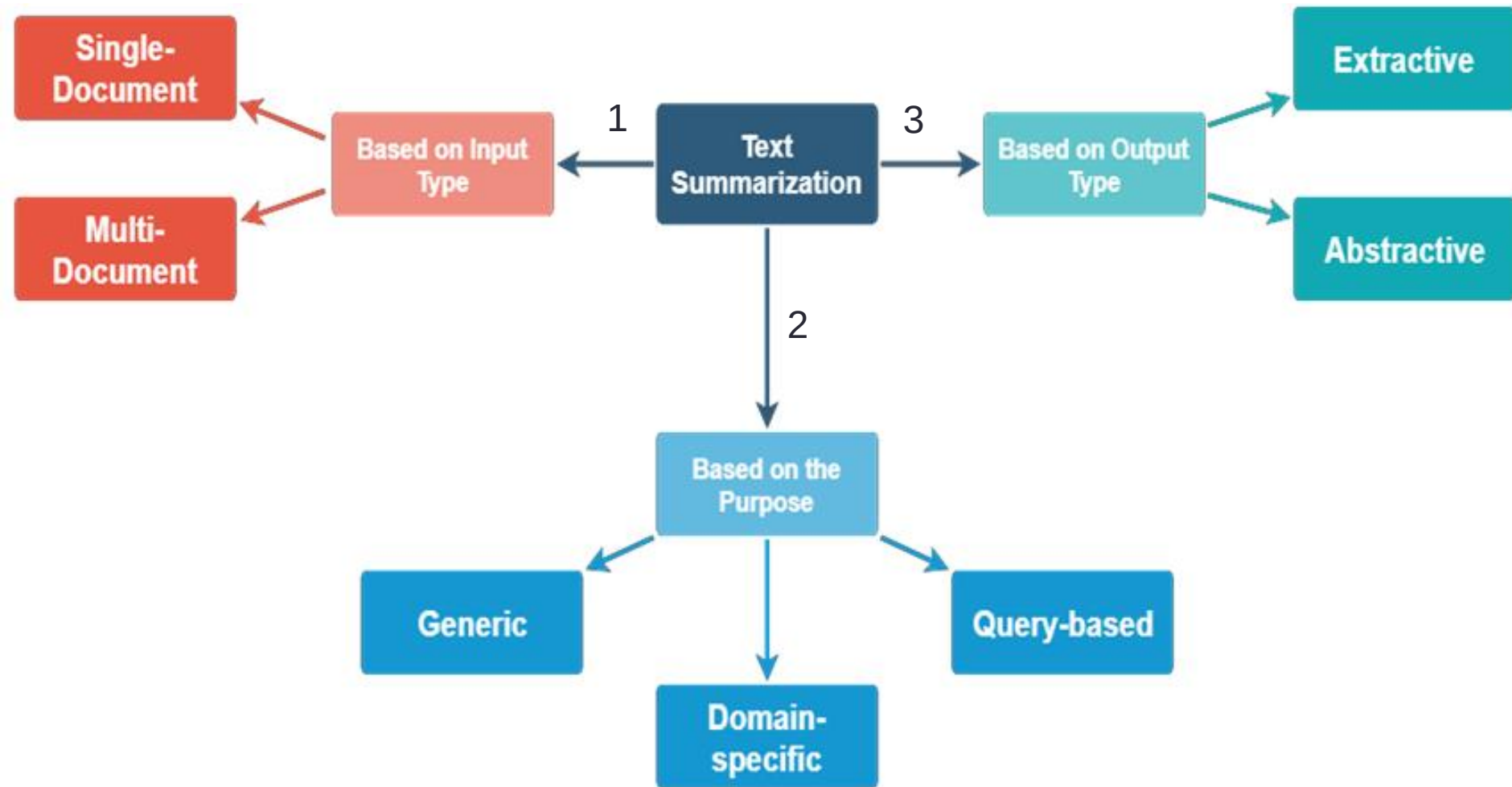
- Automatically generate a summary of an input document (or multi-documents)



Summarization Applications

- **Headlines** (from around the world)
- Minutes (of a meeting)
- Previews (of movies)
- **Reviews** (of a book, CD, movie, etc.)
- Digests (TV guide)
- Biography (resumes, obituaries)
- Bulletins (weather forecasts/stock market reports)
- Histories (chronologies of salient events)

Text Summarization Dimensions



Query-based summarization (Snippets)

Google search results for "text mining".

Google's search creates snippets summarizing a Web page for a query

AI Overview

Text mining, also known as text data mining or text analytics, is the process of extracting meaningful information and knowledge from unstructured text data using computer algorithms and techniques. It involves transforming textual data into a structured format to identify patterns, trends, and hidden relationships that might not be immediately apparent.

What Is Text Mining? | IBM

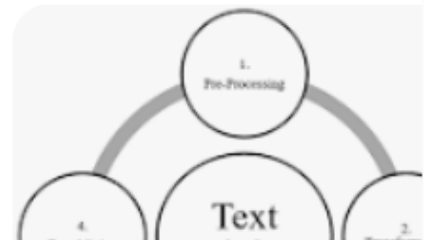
Text mining, also known as text data mining, is the process of transforming unstructured text...

Text Mining in Data Mining | GeeksforGeeks

Jan 31, 2024 — In the process of Information retrieval, we try to process the available...

Show more

Text mining, also known as text data mining, is the process of transforming unstructured text into a structured format to identify meaningful patterns and new insights. You can use text mining to analyze vast collections of textual materials to capture key



Extractive summarization

Select key sentences or phrases directly from the original text to form a summary.

House Votes to Ban Internet Taxes for 5 More Years

By LIZETTE ALVAREZ

WASHINGTON, May 10 -- In a Republican bid to woo the high-technology industry and please taxpayers, the House today rushed to the floor and then handily passed a bill to extend the current moratorium on new Internet taxes until 2006.

The moratorium, which is due to expire in October 2001, forbids states to try to find new ways of taxing Internet use, like imposing taxes on monthly access charges for Internet service providers.

The legislation passed today, which faces an uncertain future in the Senate, does not directly address the question of sales taxes; it would not stop states from trying to collect taxes for goods sold on the Internet.

By failing to address sales taxes, however, the measure alarmed some traditional retailers, as well as state governments that say they have found it nearly impossible to collect taxes for goods sold online.

"The single largest contributor to our economic prosperity has been the growth of information technology -- the Internet," said

Representative John R. Kasich, an Ohio Republican. "Why would we try to tax something, why would we try to abuse something, why would we try to limit something that generates unprecedented growth, wealth, opportunity and unprecedented individual power?"

Critics of the bill say the moratorium, while seemingly benign, ignores the thorny question of how state and local governments can best collect taxes on the billions of dollars of merchandise sold over the Internet each year. These taxes are expected to provide a crucial future source of revenue for states, especially as more consumers buy goods online.

The bill's opponents -- a consortium of retailers, small-business groups and governors -- say that consumers who buy merchandise over the Internet can easily circumvent the sales and "use" taxes that would be collected automatically if the same merchandise is bought at a bricks-and-mortar retail store.

The National Governors' Association is working on the best way to collect electronic sales tax.

Abstractive summarization

- **Sequence-to-sequence models** (e.g., RNNs/LSTMs/Transformers)
 - Trained to map input sequences (source document) to output sequences (summary).
- **Attention mechanisms**
 - Enable the model to focus on different parts of the input sequence when generating each part of the summary. This helps capture the relevant information and context for creating a coherent summary.
- **Encoder-decoder architecture**
 - The encoder processes the input sequence, the decoder generates the summary.
- **Reinforcement learning**
 - These models are trained using a combination of supervised learning and reinforcement learning, with the latter providing rewards or penalties based on the quality of generated summaries.
- **Transfer learning**
 - Fine-tuning pre-trained language models like BERT can leverage the broad linguistic knowledge encoded in the pre-trained models.

Extractive Summarization

Extractive summarization

- Simplest strategy: take the first sentence



The screenshot shows a web browser displaying the Wikipedia page for 'Text mining'. The address bar shows the URL 'https://en.wikipedia.org/wiki/Text_mining'. The page features the Wikipedia logo on the left, navigation links like 'Main page', 'Contents', and 'Featured content'. The main content area has the title 'Text mining' and a sub-header 'From Wikipedia, the free encyclopedia'. The first sentence of the article is highlighted: 'Text mining, also referred to as **text data mining**, roughly equivalent to **text analytics**, is the process of deriving high-quality information from text. High-quality information is typically derived through the devising of patterns and trends through means such as statistical pattern learning. Text mining usually involves the process of structuring the input text (usually parsing, along with the

[Text mining - Wikipedia](https://en.wikipedia.org/wiki/Text_mining)

https://en.wikipedia.org/wiki/Text_mining ▼ Traduci questa pagina

Text mining, also referred to as **text data mining**, roughly equivalent to **text analytics**, is the process of deriving high-quality information from **text**. High-quality information is typically derived through the devising of patterns and trends through means such as statistical pattern learning.

[Text analytics](#) · [Text analysis processes](#) · [Applications](#) · [Intellectual property law](#)

1. Topic Representation

- Finds key topics in the text and scores sentences based on how well they reflect those topics.

2. Indicator Representation

- Uses various features (not just topics) to estimate sentence importance, often with machine learning.

• Selecting the summary

- Sentences are chosen one by one to create the final summary.

Topic Representation Approaches

- Identifying the **main topics** of the text.
- Sentences are scored by how well they represent these topics.
- Methods include:
 - **Topic Words**: Score sentences based on **important words** and **their weights**.
 - **Lexical Chains**: Use tools like WordNet to **group semantically related words** into topics.
 - **Latent Semantic Analysis (LSA)**: Discover **patterns in word usage** that suggest topics.
 - **Bayesian Models (e.g., LDA)**: Represent text as **a mix of topics**, each with weighted word lists.

Centroid-based summarization

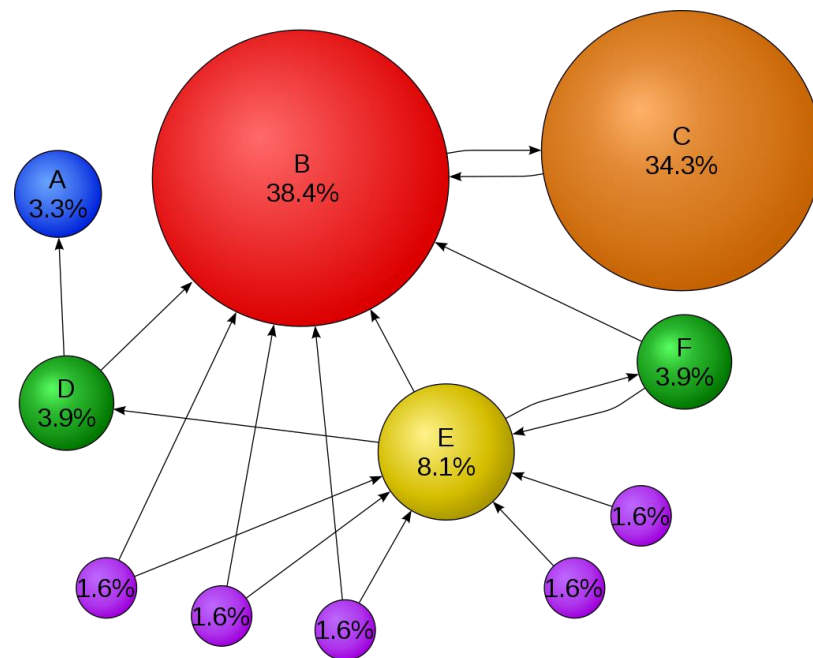
- Centroid-based summarization computes a **centroid vector**, which represents the **central meaning of a document** or cluster of documents, and selects sentences that are **most similar to this centroid** to form the summary.
- **Steps:**
 - **Preprocessing:** Tokenization, stopword removal, etc.
 - **Sentence Vectorization:** TF-IDF or embeddings.
 - **Centroid Calculation:** Average of TF-IDF vectors or weighted average of word vectors from the document or document cluster
 - **Sentence Scoring:** Compute the cosine similarity between each sentence vector and the centroid.
 - **Selection:** Rank sentences by similarity to the centroid and select top-k non-redundant ones for the summary

Pagerank

- Formula for calculating a node's weight

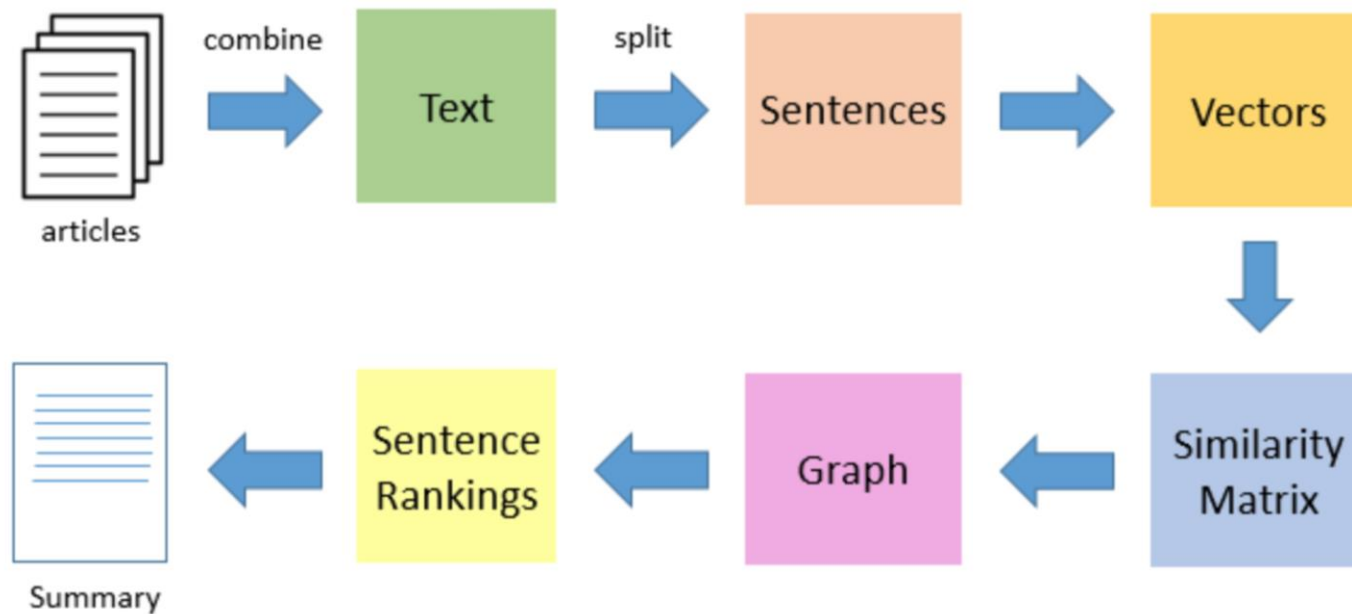
$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

- $p_1 \div p_n$: pages
- $M(p_i)$: pointer points to p_i
- $L(p_j)$: pages that p_j point to
- N : total pages



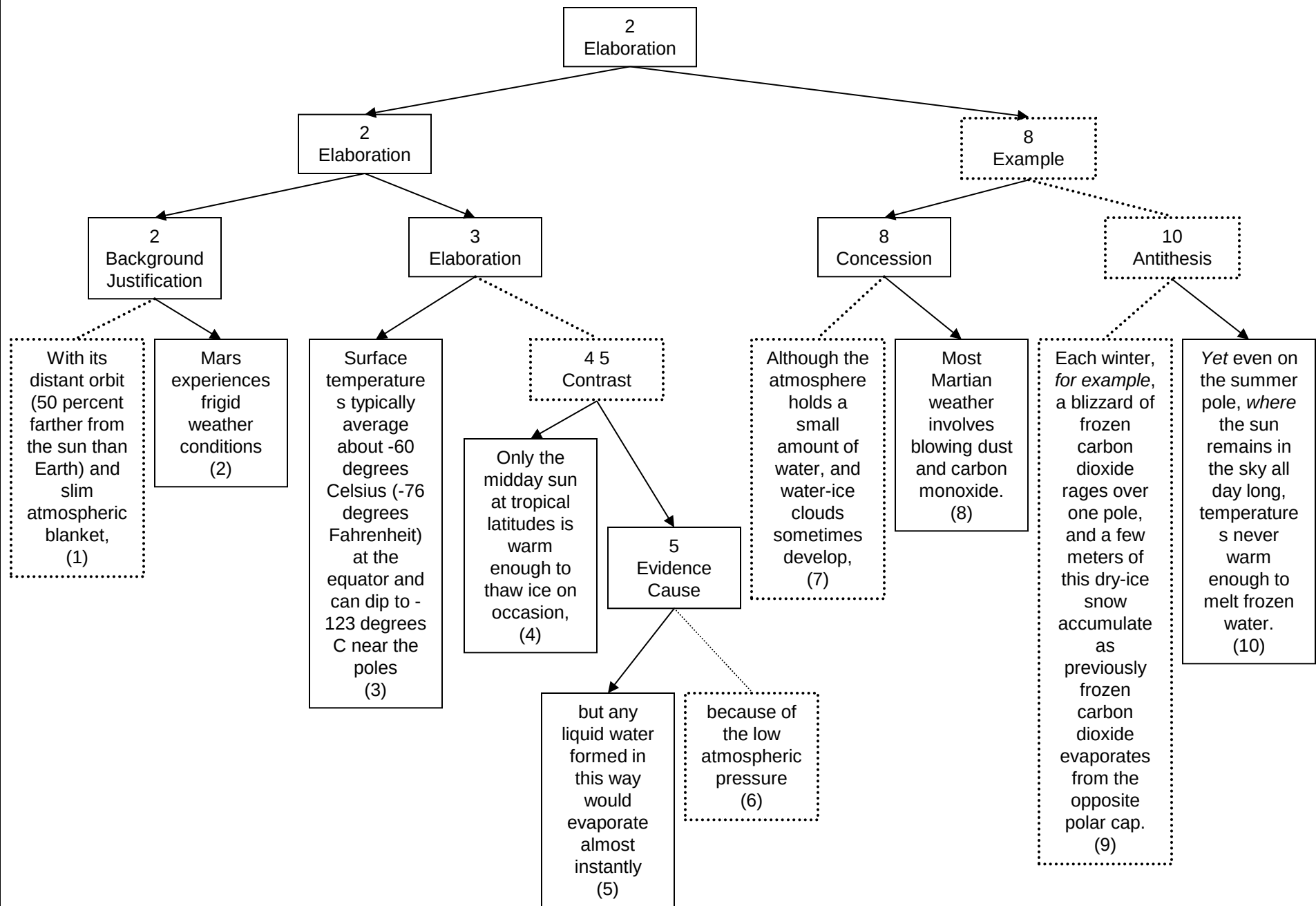
Brin, S., & Page, L. (1998). *The anatomy of a large-scale hypertextual web search engine*. Computer Networks and ISDN Systems, 30(1-7), 107-117.

TextRank & LexRank



- Each node represents a sentence.
- Edges are created based on the co-occurrence of words in the TextRank algorithm.
- **LexRank** uses the cosine similarity of **TF-IDF vectors**.
- **TextRank** uses cosine similarity based on the **number of shared words** between two sentences.

- Based on RST
- RST: Rhetorical Structure Theory, which identifies nucleus-satellite (main-support) relationships
- Example: Evidence
 - *[The truth is that the pressure to smoke in junior high is greater than it will be any other time of one's life:]**[we know that 3,000 teens start smoking each day.]*
 - Evidence: Combining Nuclear - Satellite increases the reader's trust in N
- Based on textual connections that connect the most important components in the document: 70% P and R.



Indicator Representation Approaches

- Focus on **various features** that may signal importance (not necessarily topics).
- Each sentence is evaluated using indicators such as:
 - Sentence length
 - Position in the document
 - Presence of specific keywords
 - Others (e.g., cue phrases, formatting)
- Often combined using machine learning to score importance.

Machine learning approaches

- Supervised learning approaches model the summarization as a classification problem.
 - Input: sentence's representation;
 - Output: score of the sentence (the probability of a sentence to belong to the summary)
- Features:
 - **Position of the sentence in the document:** First sentences of news are almost always informative.
 - **Position in the paragraph:** First and last sentences are often important.
 - **Sentence length**
 - **Similarity of the sentence with the document title or headings.**
 - **Weights of the words** in a sentence determined by any topic representation approach.
 - **Presence of named-entities or cue phrases** from a predetermined list

Training Dataset

Dataset	#topics	#doc	#sent	#sum
DUC 01	60	607	19,022	60
DUC 02	59	567	16,125	128
DUC 04	50	500	10,715	200
CNN		92,579		92,579
DailyMail		219,506		219,506
Vn-MDS	200	600	9,802	400
VíM	300	1,945	25,100	60
VSoLSCSum		141	3760	2448
VNDS		105,418	1,868,007	105,418

Sentence Selection Methods for Summarization

- **Top- n selection**: Choose the n most important sentences that fit the summary length.
- **Maximal Marginal Relevance (MMR)**: Pick sentences one by one, balancing importance and avoiding repetition.

Maximal Marginal Relevance

Greedy selection — at each step, pick the sentence that balances:

- Relevance to the main topic
- Dissimilarity to already selected sentences

$$\text{MMR}(S_i) = \lambda \cdot \text{Sim}(S_i, C) - (1 - \lambda) \cdot \max_{S_j \in S_{\text{selected}}} \text{Sim}(S_i, S_j)$$

where:

- S_i : candidate sentence
- C : centroid or query (document representation)
- Sim : similarity function (e.g., cosine)
- $\lambda \in [0, 1]$: trade-off between relevance and diversity

➤ Reduces redundancy while keeping key information.

A useful library for Text Summarization

- <https://pypi.org/project/sumy/>
- Implemented summarization methods:
 - Luhn heuristic method
 - Edmundson heuristic method
 - Latent Semantic Analysis
 - LexRank
 - TextRank
 - SumBasic
 - KL-Sum
 - Reduction - Graph-based summarization

- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- ROUGE is a set of metrics for comparing **generated summaries** with **reference summaries** using overlap of n-grams, word sequences, and word pairs
- Use in Summarization
 - ROUGE-N: Measures content selection (basic word overlap).
 - ROUGE-L: Longest Common Subsequence

- Measures n-gram overlap between generated and reference summaries.

$$\text{ROUGE-N}_{\text{recall}} = \frac{\sum_{S \in \text{RefSumm}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{RefSumm}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}$$

- $\text{Count}_{\text{match}}$: #overlapping n-grams between generated and reference
- Denominator: total number of n-grams in the reference

$$\text{ROUGE-N}_{\text{precision}} = \frac{\sum \text{Count}_{\text{match}}(\text{gram}_n)}{\sum \text{Count}_{\text{gen}}(\text{gram}_n)}$$

$$\text{ROUGE-N}_{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example:

- Reference: The cat is on the mat.
- Output: The cat and the dog.

Precision = overlap / output unigrams = 3 / 5 = **0.60**

Recall = overlap / reference unigrams = 3 / 6 = **0.50**

$$\text{F1 Score} = 2 \cdot \frac{P \cdot R}{P + R} = 2 \cdot \frac{0.6 \cdot 0.5}{0.6 + 0.5} = \frac{0.6}{1.1} \approx 0.545$$

- Measures the longest matching sequence of words.
- Captures sentence-level structure.
- Evaluates fluency and grammar similarity.

$$\text{ROUGE-L}_{\text{recall}} = \frac{LCS(R, O)}{|R|} \quad \text{ROUGE-L}_{\text{precision}} = \frac{LCS(R, O)}{|O|}$$

$$\text{ROUGE-L}_{F1} = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\text{Recall} + \beta^2 \cdot \text{Precision}}$$

- Where β is usually set to **1.2** to favor recall (as in summarization, recall is often prioritized).

Useful libraries for TS evaluation

- <https://pypi.org/project/sumy/>
- <https://pypi.org/project/rouge/>
- <https://pypi.org/project/py-rouge/>
- <https://pypi.org/project/rouge-score/>

Deep Learning Approaches

Extractive Summarization - Formulation

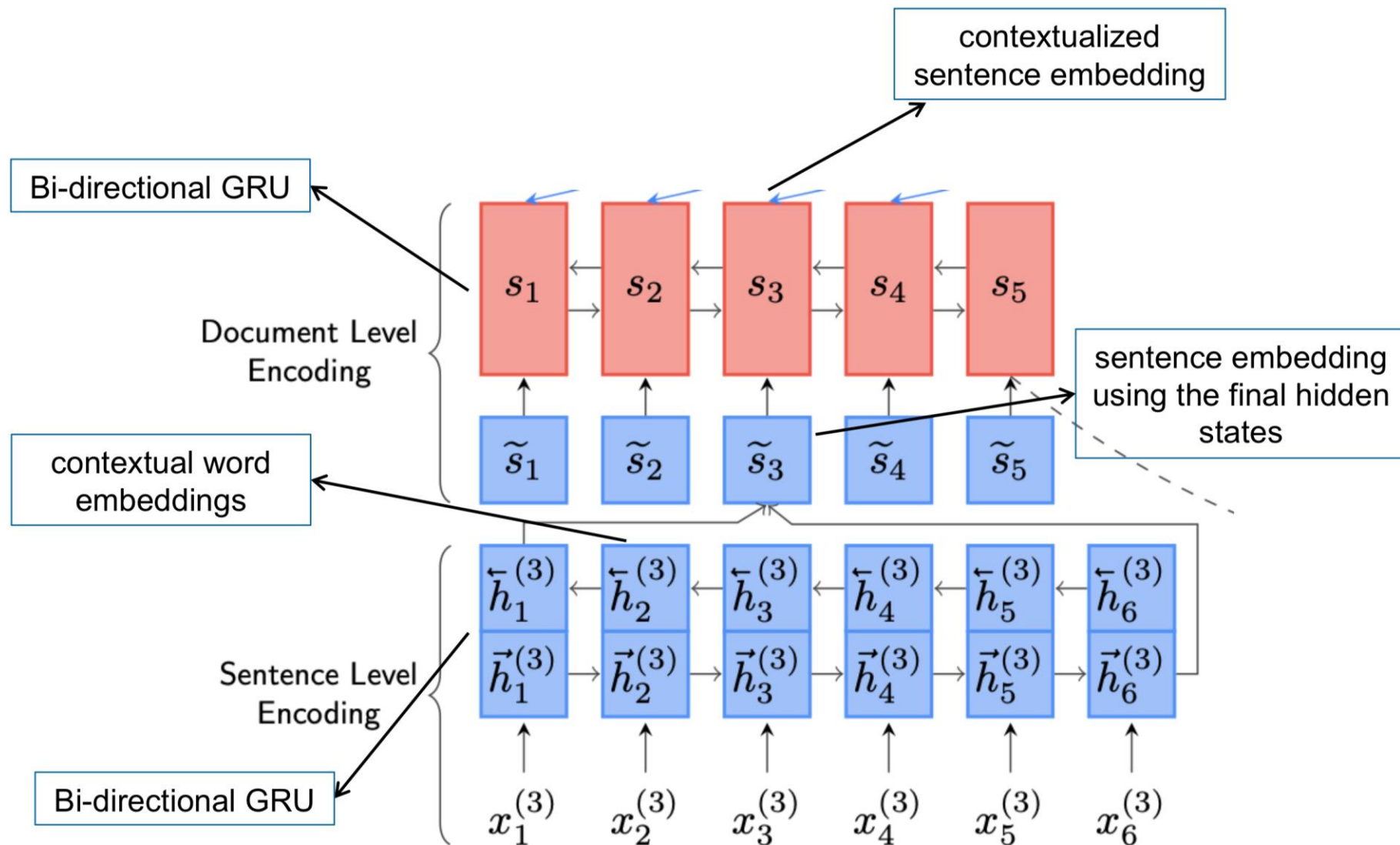
Problem definition

- Document D contains L sentences: $D = \{s_1, s_2, \dots, s_L\}$
- Each sentence s_i contains a set words, each annotated with $x^{(i)}$
- Extractive summarization objective: **select l sentences** of the document such that they provide the “*best*” summary (concise and comprehensive)

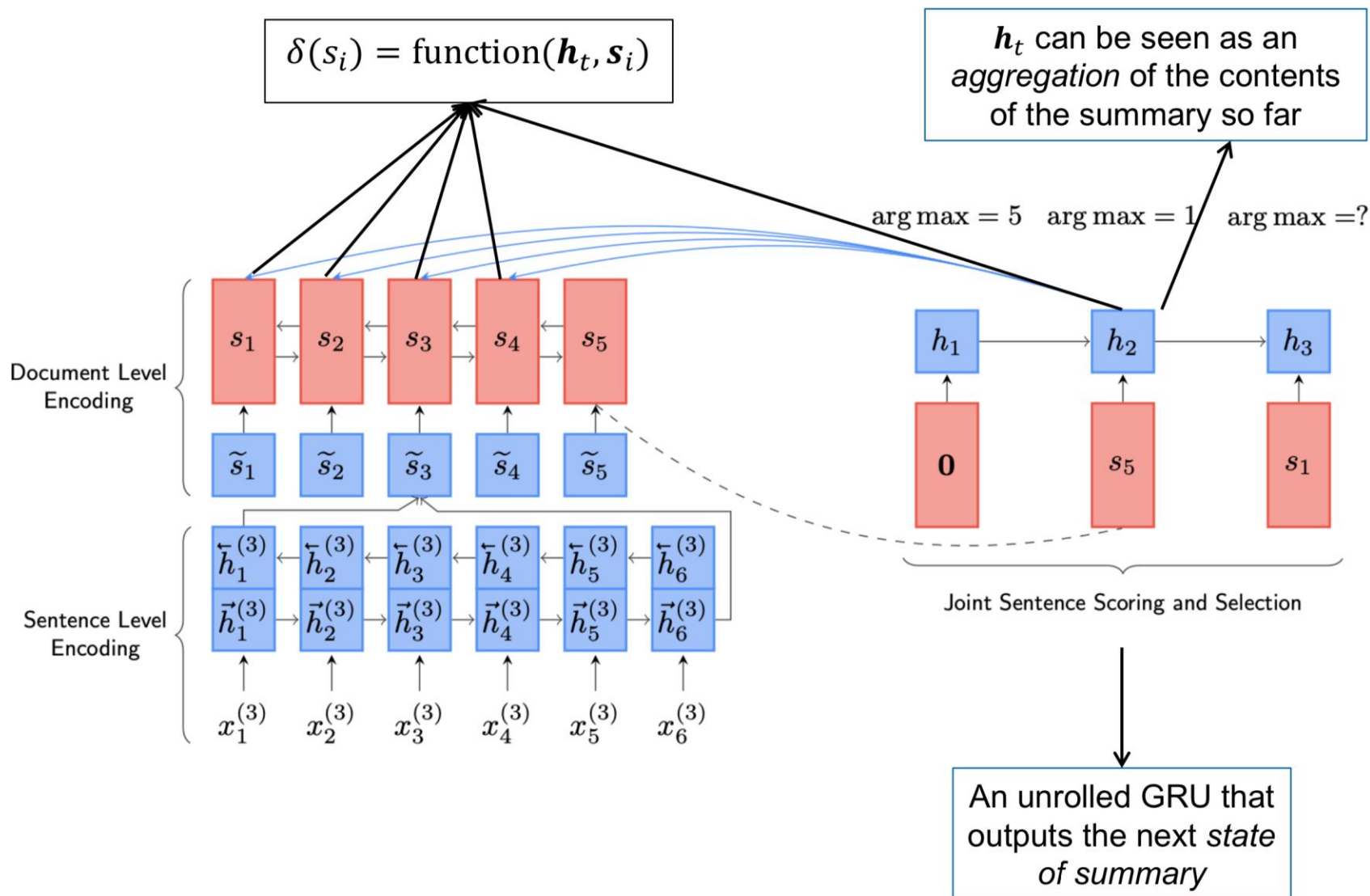
Extractive Summarization - Steps

- At each time step, the model wants to decide which sentence to include in the summary (**sentence selection**)
- To do sentence selection, at each time step, the model assigns scores to sentences that are not included in summary (**sentence scoring**), and selects the one with the highest score
- The sentence scoring is based on the **representation of each sentence**, but also the content of the **previously selected sentences**
 - *Why on previously selected sentences?* Intuitively, if some contents are already included in the summary, the model should avoid selecting the sentences with similar contents

Sentence Encoding



Sentence scoring (2)



Sentence scoring and selection

- **Sentence scoring** learns a function δ that assigns a score to each sentence s_i at time step t . It uses:
 1. sentence embedding s_i
 2. information of **previously selected sentences**, embedded in the vector $\mathbf{h}_t \rightarrow$ **current state of summary**

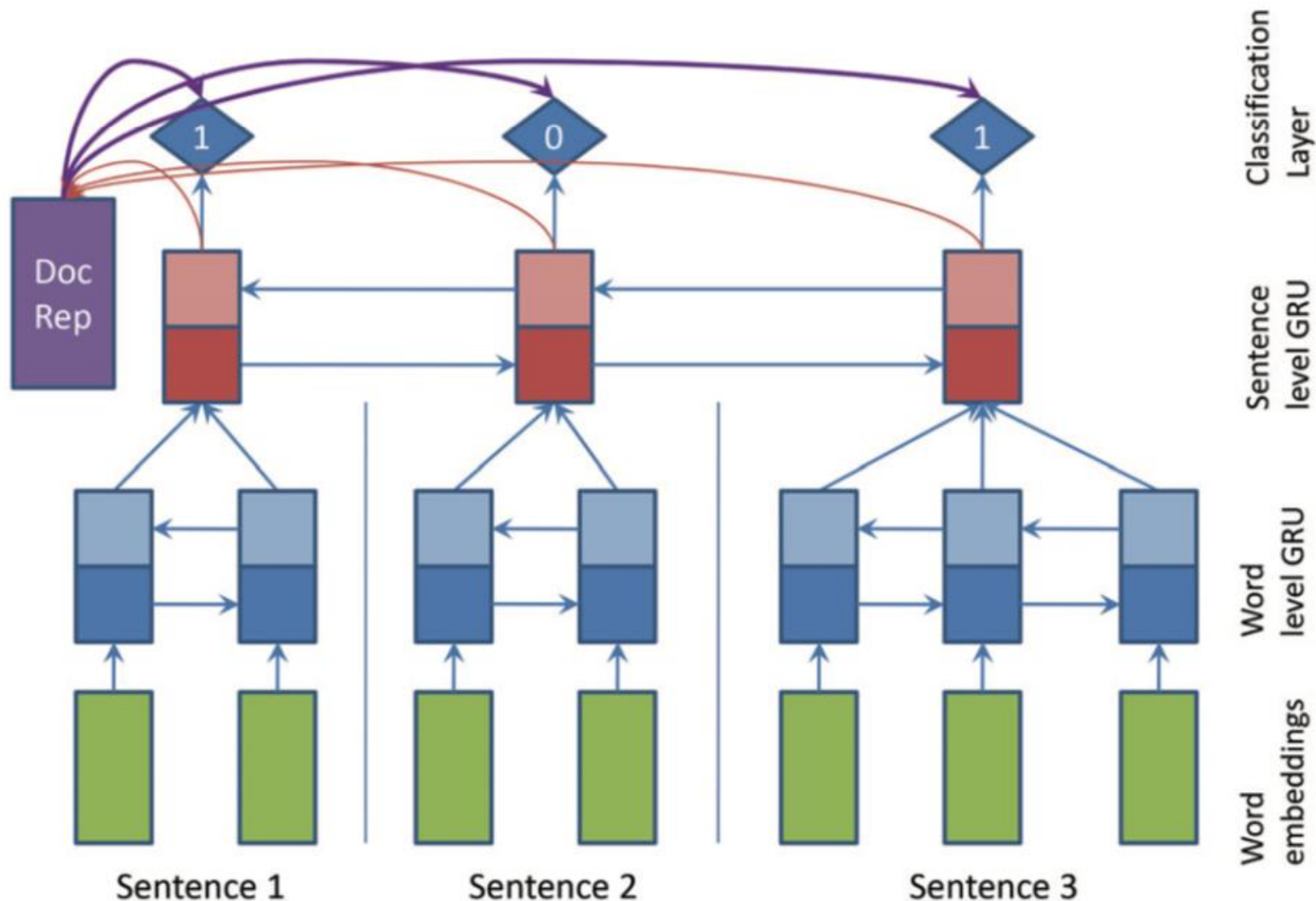
$$\delta(s_i) = \text{function}(\mathbf{h}_t, \mathbf{s}_i)$$

$$\delta(s_i) = \mathbf{w}_s \tanh(\mathbf{h}_t \mathbf{W}_q + \mathbf{s}_i \mathbf{W}_d + \mathbf{b}_i)$$

- $\delta(s_i)$ is calculated for the sentences that are **not yet included in the summary**
- The sentence with **highest** δ is added to summary

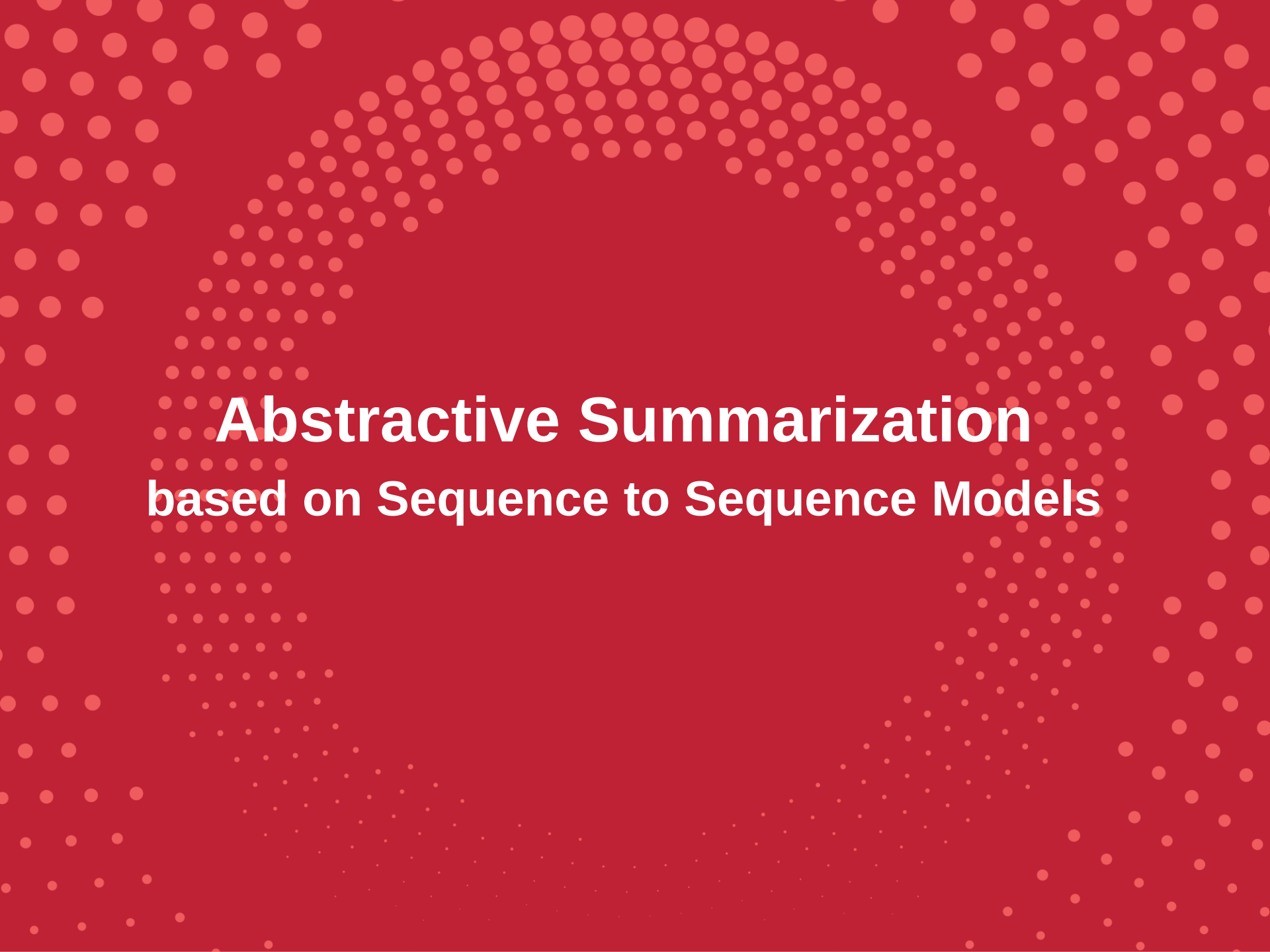
Sequence Model for extractive summarization

SummaRuNNer: A Recurrent Neural Network based Sequence Model for Extractive Summarization of Documents – AAAI 2017, Nallapati et al.



Sequence Model for extractive summarization (2)

- Each sentence is represented by averaging the word embeddings (using Word2Vec or GloVe) in the sentence.
- Sentence representations are fed into a Bidirectional GRU network
- **Document Representation:** computed by averaging the hidden vectors of all sentences.
- Each sentence is scored based on:
 - **Importance:** Similarity between the sentence and the document representation.
 - **Novelty:** Dissimilarity between the sentence and already selected sentences.
 - **Position:** The position of the sentence in the document.
- The final score is passed through a sigmoid function to produce a probability of selecting the sentence into the summary.

The background is a solid red color with a pattern of small, light red dots. These dots are arranged in a way that they form a large, faint circular shape in the center of the slide, with the density of the dots being higher in the center and fading out towards the edges.

Abstractive Summarization **based on Sequence to Sequence Models**

Abstractive Summarization

- An example from CNN/DailyMail

Document

SAN FRANCISCO, California (Reuters) -- Sony has cut the price of the PlayStation 3 by \$100, or 17 percent, in the United States, a move that should boost the video game console's lackluster sales.

Starting Monday, the current PS3 60 gigabyte model will cost \$499 -- a \$100 price drop.

The PlayStation 3, which includes a 60-gigabyte hard drive and a Blu-ray high-definition DVD player, will now cost \$500, or \$20 more than the most expensive version of Microsoft's Xbox 360.

The PS3 still costs twice that of Nintendo's Wii console, whose \$250 price and motion-sensing controller have made it a best-seller despite its lack of cutting-edge graphics and hard disk.

"Our initial expectation is that sales should double at a minimum," Jack Tretton, chief executive of Sony Computer Entertainment America, said in an interview.

"We've gotten our production issues behind us on the PlayStation 3, reaching a position to pass on the savings to consumers, and our attitude is the sooner the better."

...

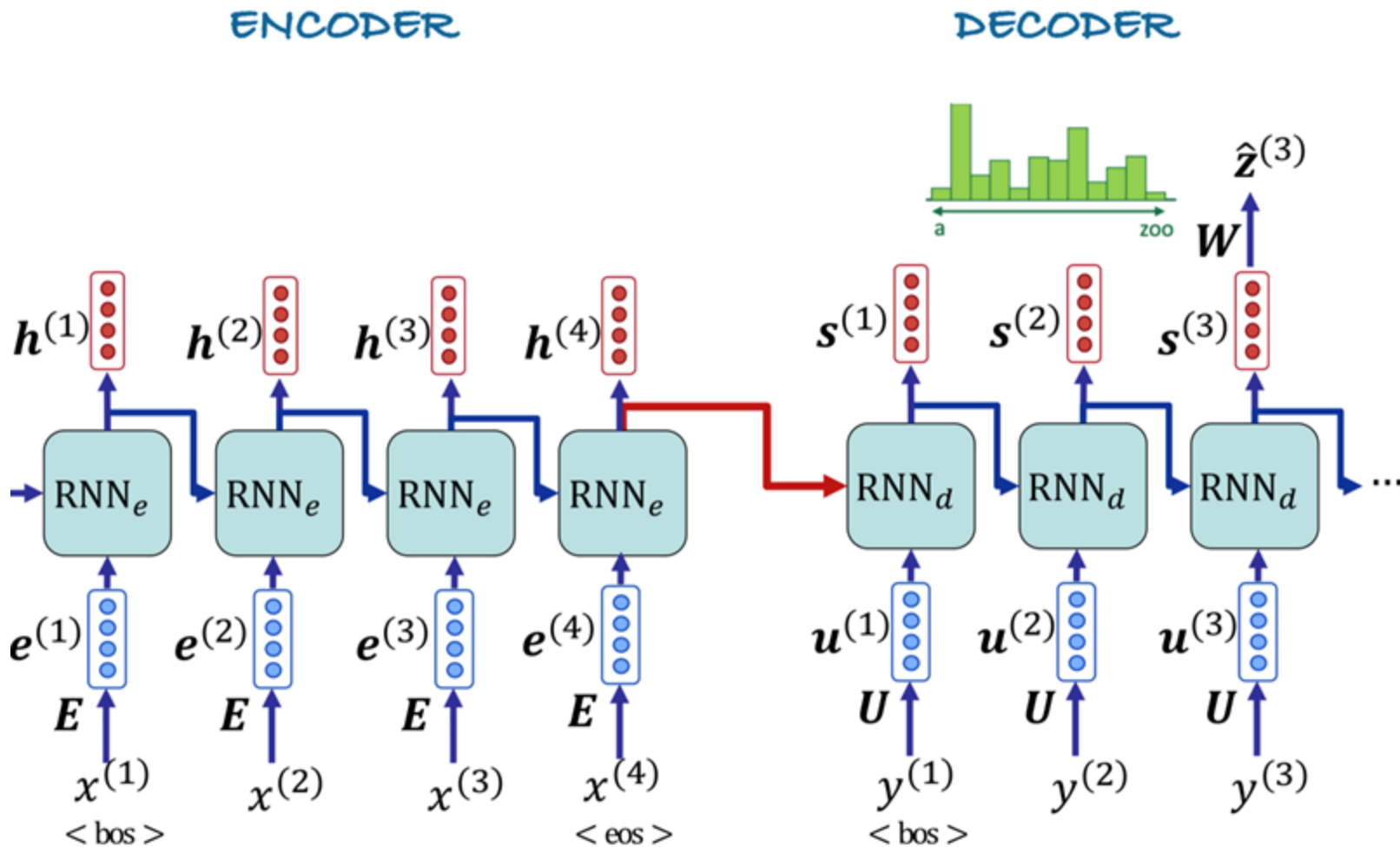
Summary

- Sony drops price of current 60GB PlayStation 3 console by \$100 in U.S.
- PS3 still costs twice that of Nintendo's best-selling Wii console, which is \$250
- Some expect Microsoft to respond with its first price cuts on the Xbox 360
- Sony to revise PS3 console with bigger 80GB hard drive

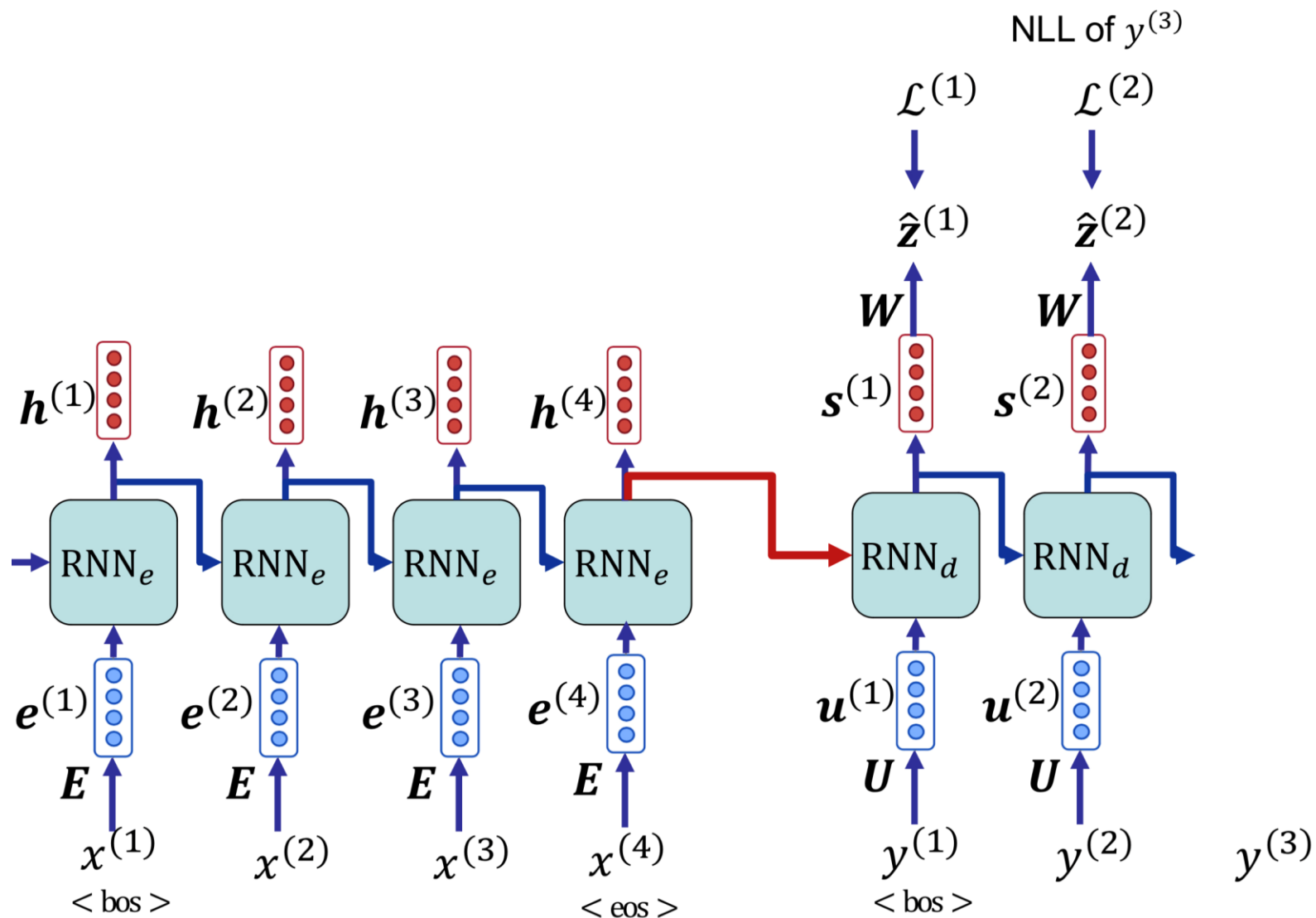
Seq2Seq Steps

- Like Language Modeling, we ...
- ... **design** a model that predicts the **probabilities of the next words** of the target sequence, one after each other (in **auto-regressive** fashion): $P(y^{(i)} | X, y^{(1)}, \dots, y^{(i-1)})$
- We **train** the model by **maximizing** these probabilities for the correct next words, appearing in training data
- At inference time (or during **decoding**), we use the model to generate new target sequences, that have high **generation probabilities**: $P(Y|X)$

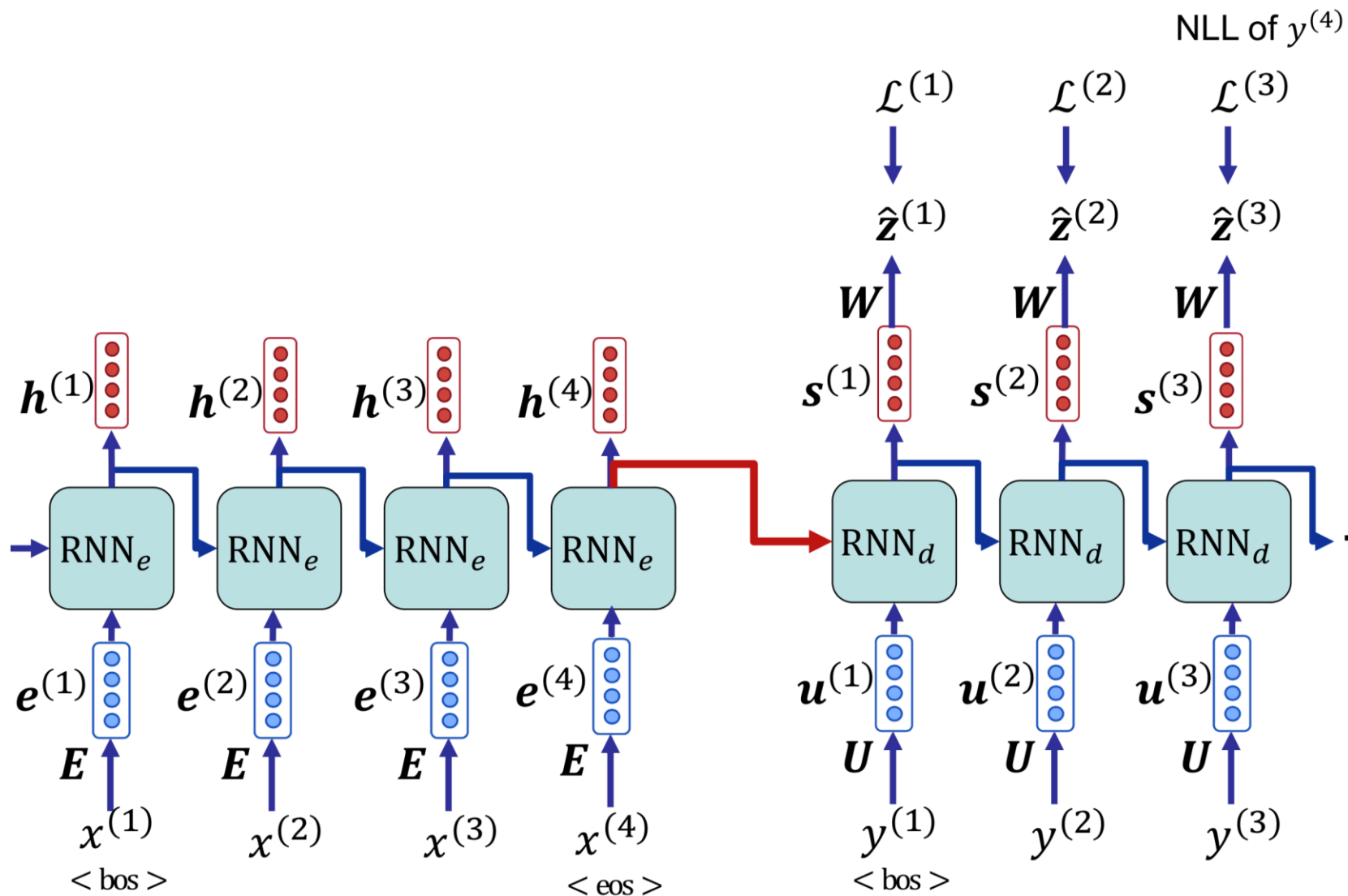
Seq2Seq with 2 RNNs



Seq2Seq – Training



Seq2Seq – Training



- **Problems in Previous Seq2Seq Summarization Models**
 - **Hallucinated Information:** Models may generate inaccurate or non-existent content.
 - **Repetition:** Generated summaries often repeat words or phrases.

Solution: Pointer-Generator Network + Coverage Mechanism

- **Pointer-Generator Network:**
 - Allows the model to choose between **generating** words from the vocabulary or **copying** words from the source text, helping handle OOV words and reduce hallucination.

$$P(w) = p_{gen} \cdot P_{vocab}(w) + (1 - p_{gen}) \cdot \sum_{i:w_i=w} a_i^t$$

- p_{gen} : decide whether to generate or copy a word

Seq2Seq model with attention for abstractive summarization

- **Coverage Mechanism:** Tracks attention history to discourage repeating the same content.

1. **Tracking Past Attention:** At each decoding step t , a coverage vector c^t is maintained:

$$c^t = \sum_{t'=0}^{t-1} a^{t'}$$

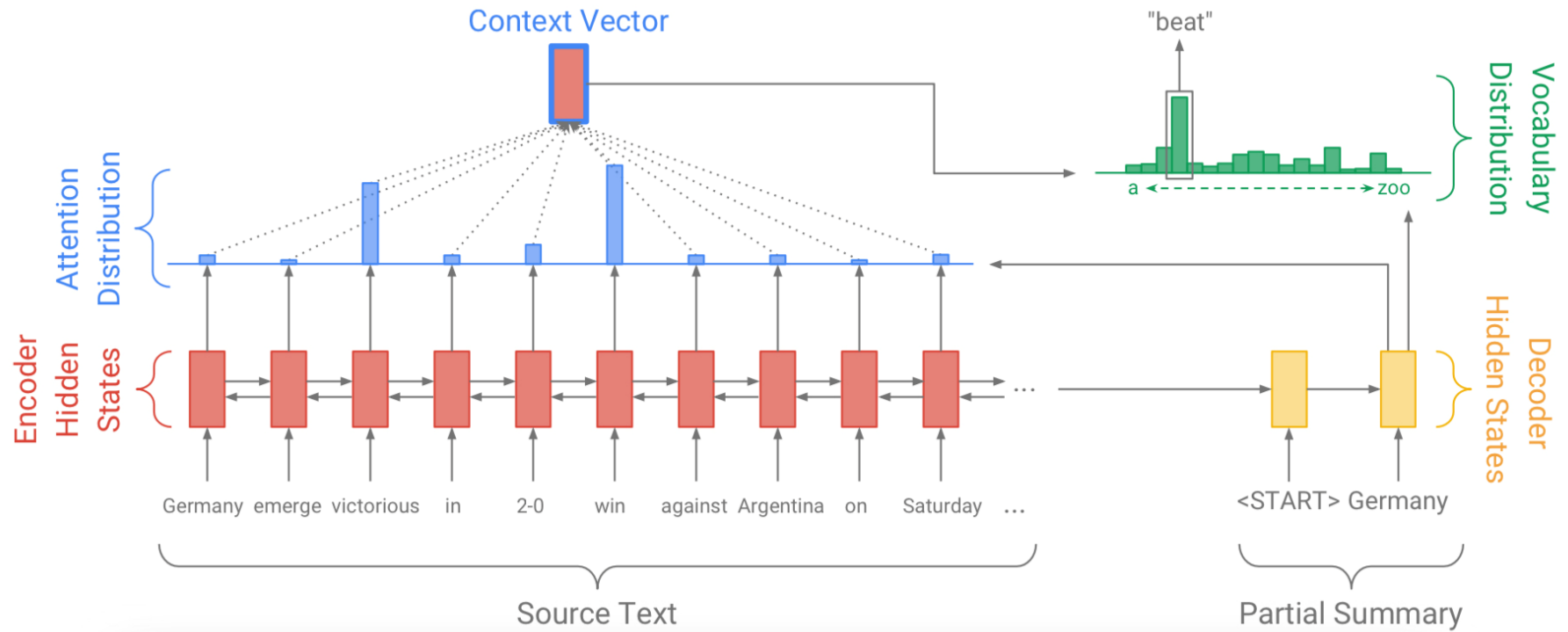
c^t : cumulative attention, it tells the model what has already been summarized.

2. **Computing Coverage Loss:** encourages the model not to focus too much again on already attended areas:

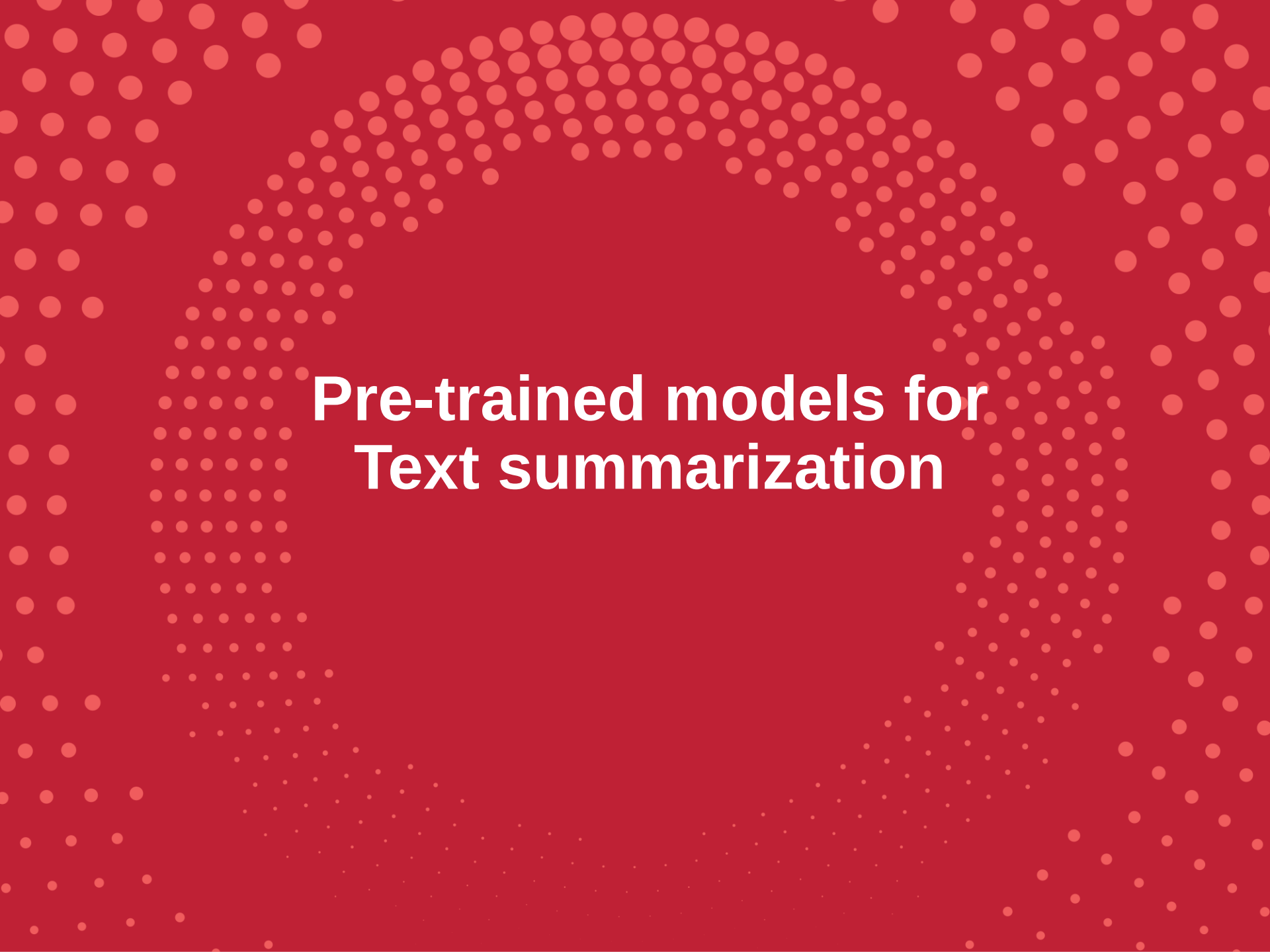
$$\text{covloss}_t = \sum_i \min(a_i^t, c_i^t)$$

- Penalizes the model if the current attention a_i^t is high in places where the coverage c_i^t is already high.
 - Learns to spread its attention more evenly and avoid summarizing the same content multiple times.

Seq2Seq model with attention for abstractive summarization

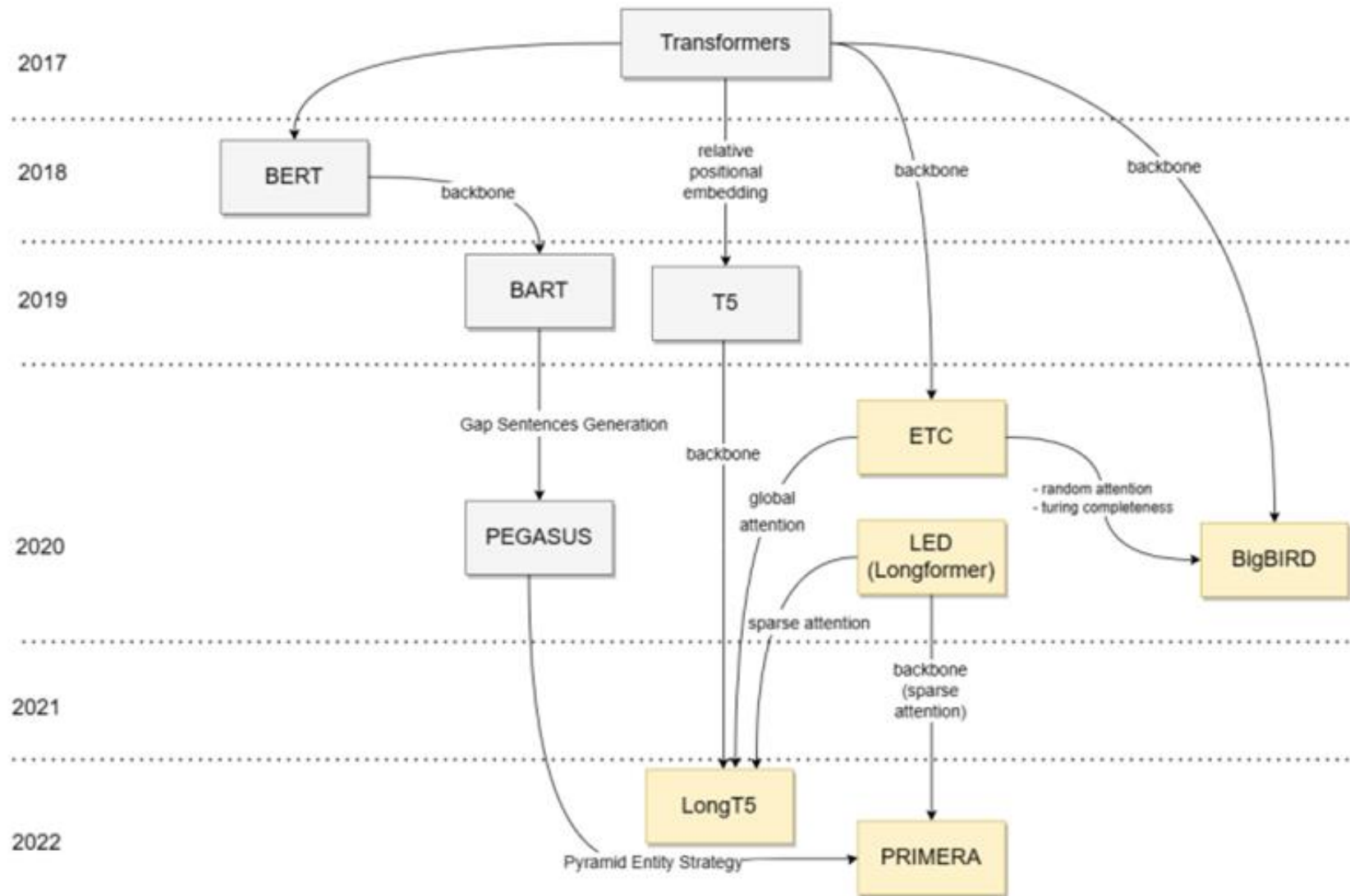


Get To The Point: Summarization with Pointer-Generator Networks – See et al. - ACL 2017



Pre-trained models for Text summarization

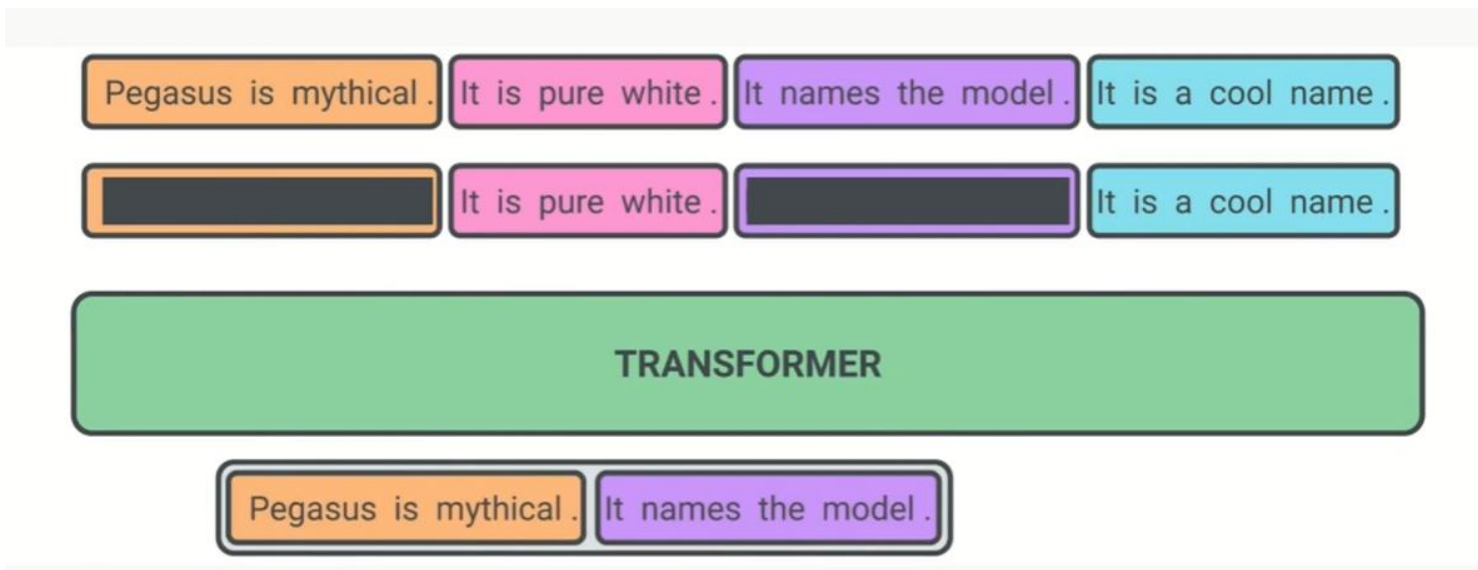
Pre-trained models for English Text summarization



Pre-trained models for English Text summarization

Model	Architecture	Dataset {R-1, R-2, R-L} - F1
PEGASUS [19]	Transformer + Gap-Sentence Selection	CNN/DailyMail (44.17, 21.47, 41.11) Multi-News (47.52, 18.72, 24.91) Gigaword (39.12, 19.86, 36.24) arXiv (44.21 16.95 38.83)
BART _{LARGE} [6]	Bidirectional encoder + autoregressive decoder	XSum (45.14 22.27 37.25)
LED _{LARGE} (16K) [29]	Transformer + windowed attention	arXiv (46.63 19.62 41.83)
BigBird-PEGASUS	LED + sparse attention (encoder side) + random attention mask PEGASUS pretraining strategy	arXiv (46.63 19.02 41.77) PubMed (46.32 20.65 42.33) BigPatent (60.64 42.46 50.01) CNN/DailyMail (43.41 20.99 40.77)
T5 _{LARGE} [18]	Transformer + relative token positioning + layer norm bias and normalization changes PEGASUS pretraining strategy	Multi-News (47.48 18.60 24.31) BigPatent (67.05 52.24 58.70) arXiv (45.86 18.40 41.62) PubMed (48.94 22.92 45.4)
PRIMERA _{LARGE} [1]	Longformer + Entity Pyramid Strategy	arXiv (47.60, 20.80, 42.60) Multi-News (49.90, 21.10, 25.90)
LongT5 _{LARGE} [32]	T5 + attention from LED	CNN/DailyMail (42.49 20.51 40.18) BigPatent (70.38 56.81 62.73) arXiv (48.28 21.63 44.11) PubMed (49.98 24.69 46.46)

- Pre-training with Extracted Gap-sentences for Abstractive Summarization
- Use a novel **pretraining objective** called **Gap Sentence Generation**
- **Method:**
 - Remove important sentences (those with high ROUGE overlap with the rest of the text).
 - The remaining document (with the gap sentences removed) is fed as the **input**.
- **Target:** The **gap sentences** become the **target output**, like a summary.



- A **multi-document summarization** model that uses **unsupervised pretraining** to help it understand how to generate coherent and informative summaries from multiple sources
- Pretrained using a **masked sentence reconstruction objective** on large-scale single-document datasets (e.g., CNN/DailyMail), but designed to simulate **multi-document summarization**.

PRIMERA - multi-document summarization

Key Steps:

1. Input Construction:

- From a single document, PRIMERA **splits sentences** into two groups and **shuffles** them to mimic multiple sources.
- This simulates MDS input: multiple documents with mixed sentence order.

2. Masked Sentence Prediction:

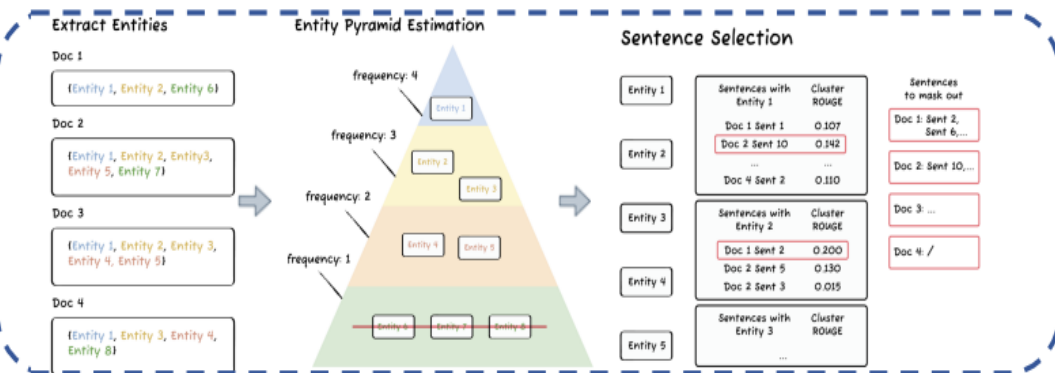
- Randomly **mask entire sentences** and train the model to **reconstruct** them based on surrounding context.
- This encourages the model to learn **inter-sentence coherence**, **discourse structure**, and **information selection**.

3. No Need for Human Summaries:

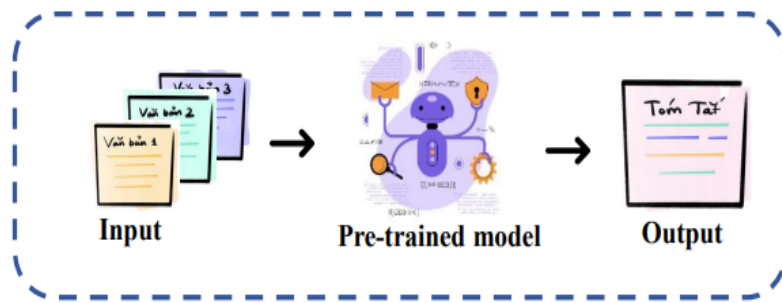
- Unlike standard summarization training, PRIMERA **does not require gold summaries** during pretraining.
- This makes the model scalable and suitable for low-resource domains.

Unlabeled Multi-document dataset

Pre-processing step - Novel Masking Strategy: Entity Pyramid

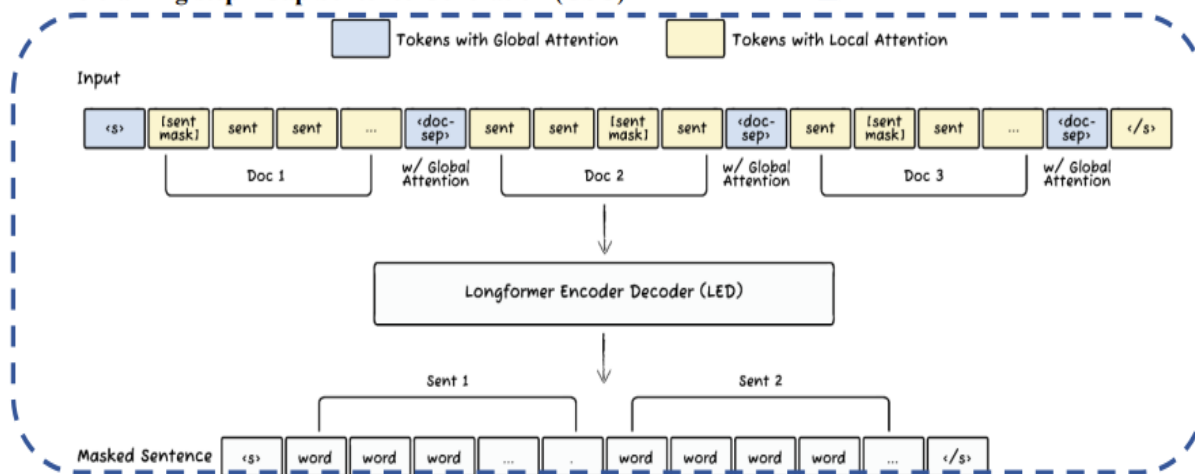


Fine-tuning + Inference step



Pre-training step - Gap Sentence Generation (GSG)

Multi-document dataset with pseudo summary



Uses a **pretraining strategy similar to BERT**, but is designed to handle **much longer documents** by using an **efficient attention mechanism** called **sliding window attention**.

1. Training Objective: Masked Language Modeling (MLM)

2. Input Length: Up to 4,096 tokens

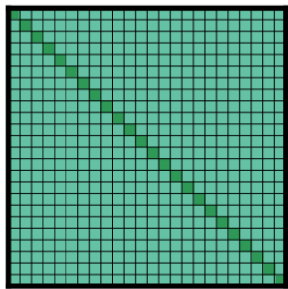
3. Efficient Attention: Sliding Window + Global Attention

- Uses **sliding window attention** to reduce the quadratic complexity of full self-attention.
- Adds **global attention** tokens for special cases (e.g., [CLS] or question tokens) to allow full-context interaction when needed.

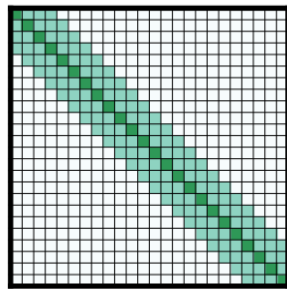
4. Pretraining Data: Pretrained on large-scale datasets, similar to RoBERTa

Sliding Window

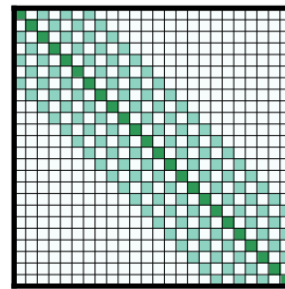
- Instead of attending to **all tokens** (as in full self-attention, which scales quadratically), each token only attends to a **local neighborhood** of tokens within a fixed window.
- For each token at position i , the model only computes attention with tokens in the range $[i-w, i+w]$ where w is the **window size** (e.g., 256 tokens).
- This means each token "sees" only a limited number of nearby tokens, making computation **linear** in sequence length rather than quadratic.



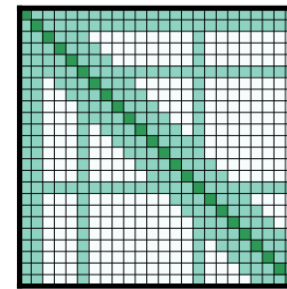
(a) Full n^2 attention



(b) Sliding window attention

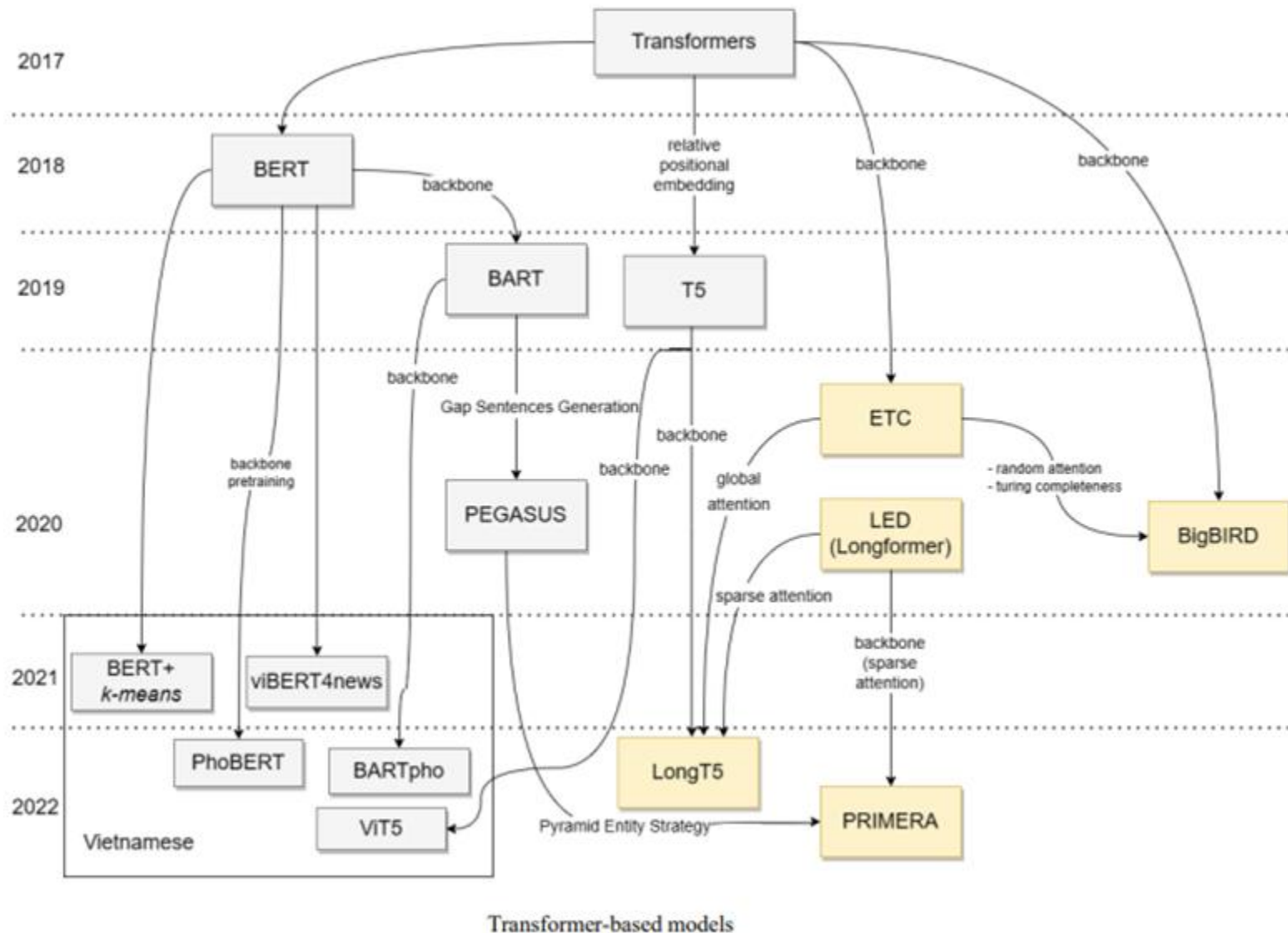


(c) Dilated sliding window



(d) Global+sliding window

Pre-trained models for Vietnamese Text summarization



Pre-trained models for Vietnamese Text summarization

Models	WikiLingua			Vietnews		
	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-1	ROUGE-2	ROUGE-L
Transformer (RND2RND)	46.25	16.57	29.82	57.56	24.25	35.53
PhoBERT2PhoBERT	50.4	19.88	32.49	60.37	29.12	39.44
mBERT2mBERT	52.82	20.57	31.55	59.67	27.36	36.73
mBART	55.21	25.69	37.33	59.81	28.28	38.71
mT5	55.27	27.63	38.30	58.05	26.76	37.38
BARTpho	57.16	31.18	40.89	61.14	30.31	40.15
ViT5 _{base} 256-length	57.86	29.98	40.23	61.85	31.70	41.70
ViT5 _{base} 1024-length	<u>58.61</u>	<u>31.46</u>	<u>41.45</u>	<u>62.77</u>	<u>33.16</u>	<u>42.75</u>
ViT5 _{large} 1024-length	60.22	33.12	43.08	63.37	34.24	43.55

Test result on Wikilingua and Vietnews Summarization